

Seat No.

43



VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Authorization Seal

(P8)

Test : CAT- I / CAT- II / Mid-Term

Register No.: 22BCE0023

Course Code: BCSE202L Class NBR: 6331

Student's Name: Mr/Ms. Shaan Jain

Course Title: Data Structures & Algorithms

Programme : B.Tech School : SCOPE

Faculty Name : Prof. : Priyanka N

Slot : A1+TA1 Session : FN/AN/EVN

Date of Exam : 7-05-2023 No. of pages written :

Signature of the Student

Signature of the Invigilator :

Name / Emp. ID of the Invigilator :

Priya
PRIA DAS
(20097)

Q No.	Marks		Q No.	Marks		Q No.	Marks		Q No.	Marks		Q No.	Marks	
	a	b		a	b		a	b		a	b		a	b
1	8		9			17			25			33		
2	7		10			18			26			34		
3	7		11			19			27			35		
4	7		12			20			28			36		
5	8		13			21			29			37		
6			14			22			30			38		
7			15			23			31			39		
8			16			24			32			40		
Sub-Total	37		Sub-Total			Sub-Total			Sub-Total			Sub-Total		

Grand Total Marks :

37

Evaluator's Signature

Date :

Evaluator's Name : Prof.

Marks In Words

Q1

Given: $A[] = A \rightarrow B \rightarrow A \rightarrow B \rightarrow A \rightarrow B \rightarrow \text{NULL}$

The linked list are constructed by
struct Node

```
{  
    char int data;  
    Node *next;  
    Node (int char x)  
    {  
        data = x;  
        next = NULL;  
    }  
}
```

{ # implemented in C++ }

∴ void sepList (Node *head)

~~int~~ i = 0;

Node *temp1;

Node *temp2, *curr;

~~while~~ {

*curr = head;

while (curr->next != NULL)

{
 if (i % 2 == 0)

temp1 = ~~new~~ (Node*) malloc (sizeof (Node));

temp1->data = curr->data;

temp1->next = head;

~~temp1~~

}

else

{

temp2 = new Node (curr->data)

temp2->next = head;

}

curr = curr → next;

}

Output

Two separate linked lists with similar characters is formed.

temp 1 = [A] → [A] → [A] → NULL

temp 2 = [B] → [B] → [B] → NULL

```
(b) int compLL (Node * head1, Node * head2, int n1, int n2)
{
    int count = 0; n1 = 0, n2 = 0;
    Node * t1 = head1;
    Node * t2 = head2;
    while (t1 != head1 && t2 != head2)
    {
        if (t1 → data == t2 → data)
        {
            count++;
        }
        t1 = t1 → next;
        t2 = t2 → next;
    }
    if (count == n1 && count == n2)
    {
        return 1; // True, the lists are identical
    }
    else
    {
        return 0; // False, the lists are not identical
    }
}
```


Q2 The quicksort algorithm will have to be modified by importing `<string>` header file to sort strings in lexicographical order.

The following changes are needed to be implemented:-

```
char int partition ( int arr, int l, int h)
{
    int char pivot [50] = arr[h];
    int i = l-1;
    for (int j = l ; j <= h-1 ; j++)
    {
        if (strcmp(arr[j], pivot) < 0)
        {
            i++;
            swap (arr[i], arr[j]);
        }
    }
    swap(arr[i+1], arr[h]);
    return i+1;
}
```

```
void qsort ( arr[], int l, int h)
{
    if (l < h)
    {
        int p = partition (arr, l, h);
        qsort (arr, l, p-1);
        qsort (arr, p+1, h);
    }
}
```

★ Illustration the modified QuickSort algorithm

Smith	Johnson	Williams	Brown	Jones	Garcia	Miller	Davis
-------	---------	----------	-------	-------	--------	--------	-------

$h=7$

$l=0$

$\{B, D, G, J, M, S, W\}$

$P=1$

$QS(2, 7)$

$QS(0, 0)$

Brown

$QS(P+1=0+1, h=1)$

$\therefore \{Brown\}$

Williams	Garcia	Smith	Johnson	Jones	Miller
----------	--------	-------	---------	-------	--------

$h=7$

$l=2$

$\{G, J, M, S, W\}$

$P=5$

$QS(6, 7)$

$QS(2, 4)$

Garcia	Johnson	Jones
--------	---------	-------

$l=2$

$h=4$

$\{Garcia, Johnson, Jones\}$

Smith	Williams
-------	----------

$\{Smith, Williams\}$

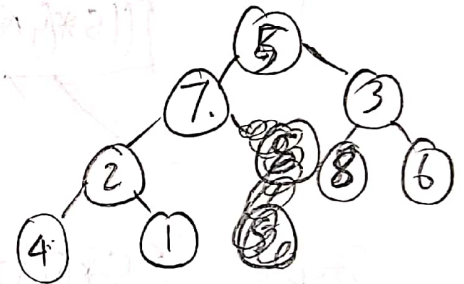
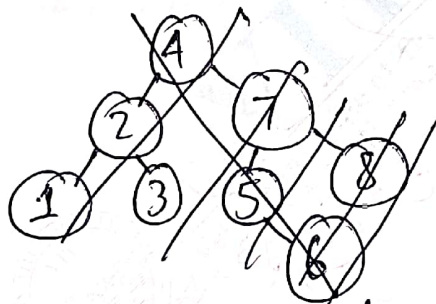
They are in correct order now

$\therefore$ Output : $\{Brown, Garcia, Johnson, Jones, Smith, Williams\}$

Extra Space

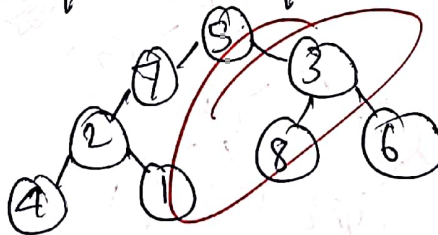
Q3 a)

{4, 2, 1, 7, 5, 8, 3, 6}

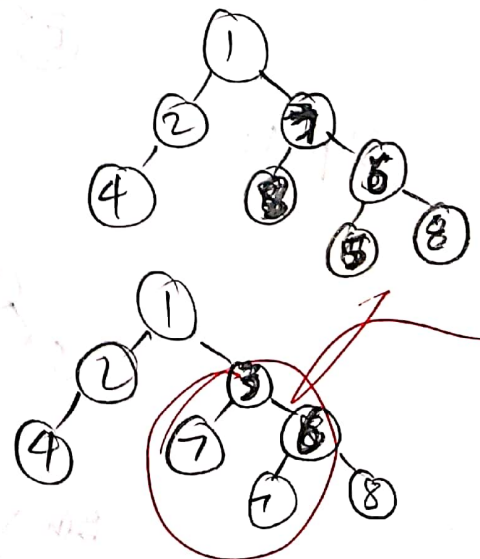
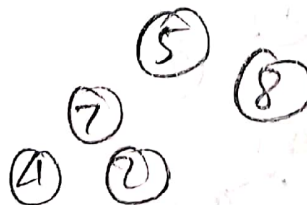


④ is leftmost leaf node

is

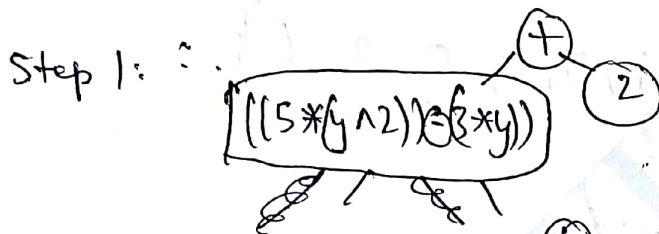


b)

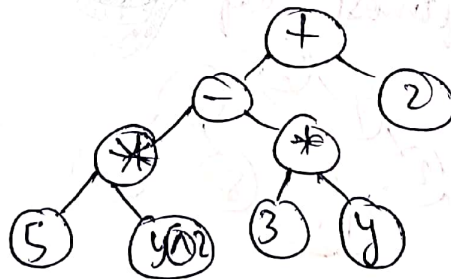


Q3 (b) $((5 * (y \wedge 2)) - (3 * y)) + 2$

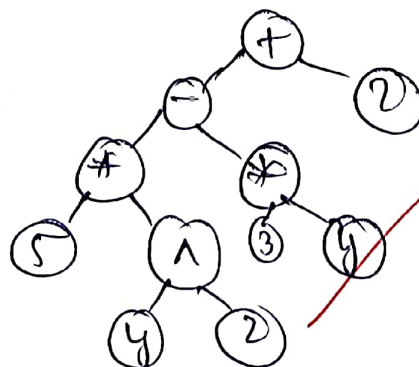
Algorithm: Least precedence operators come near the top
 following are higher precedence operators
 & operands ~~are~~ tend to be leaf nodes.



Step 3:



Step 4:



Binary Expression Tree Completed

postfix expressions : 5 y 2 ^ * 3 y * - 2 +
prefix expression : + - * 5 ^ y 2 * 3 y 2

Q4 (a)

~~Algorithm:~~

We need to find the 3rd smallest element of a BST.

This can be implemented by the following code/algorithm:

```
int count = 0;
```

```
struct * node KthSmallest (struct Node * root, int k)
```

```
{  
    if (root == NULL) {  
        return NULL;  
    }
```

```
    struct Node * left = KthSmallest (root->left, k);
```

```
    if (left != NULL)
```

```
    {  
        return left;
```

```
    }  
    count ++;
```

```
    if (count == k)
```

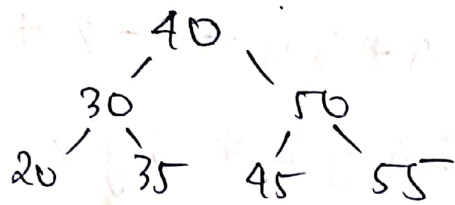
```
    {
```

```
        return root;
```

```
        return KthSmallest (root->right, k);
```

```
    }
```

✓ The time complexity (average) of the algorithm = $O(h) = O(\log n)$
where h = height of the tree.



the minimum value element will be present in the leftmost leaf node of the Binary Tree

we will reach there ~~to~~ because of the line

$\text{left} = \text{KthSmallest}(\text{root} \rightarrow \text{left}, 3);$

If $(\text{left} \neq \text{NULL})$

count++

→ we get the height of BT in count

If count == k
(3 == 3)

then print the given age that is the answer.

Q4 (b) Algorithm to check whether a given tree is BST or not: -

```

void traverseInorder(struct node *root)
{
    if (root == NULL) {
        return;
    }
    else if (root != NULL) {
        traverseInorder(root->left);
        printf("%d ", root->data);
    }
}
  
```

```

    traverseInOrder ( root → right );
}
}
int isBST ( struct node * root )
{
    static struct node * prev = NULL;
    if ( root != NULL )
    {
        if ( !isBST ( root → left ) )
            return 0;
        if ( prev != NULL && root → data <= prev → data )
            return 0;
        prev = root;
        return isBST ( root → right );
    }
    else
    {
        return 1;
    }
}
printf ( "%d", isBST ( root ) );

```

Time Complexity = $O(n)$
 Space Complexity = $O(n)$

Q5

~~The~~ Min Heap : A heap (~~is~~ complete binary tree) with smallest elements on top & largest elements at bottom.
(root)
(leaves)

Given array: [25, 32, 40, 19, 60, 55, 22]

① Construction of Min Heap

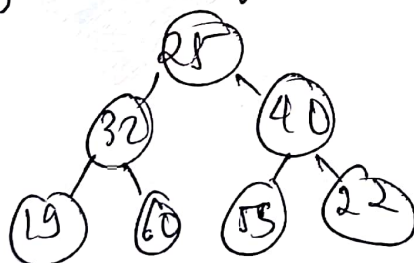
(i) Take 25 as root & add 32 to the left



(ii) Then add 40 to the right of 25 in Complete BST order



(iii) Repeating the steps above we get a Binary Tree as follows:



Now we will heapify the given tree using the following code: -

```
void MinHeapify(int i)
{
    int l = left(i), rt = right(i);
    int smallest = i;
    if (l < size & arr[l] < arr[i])
        smallest = l;
```

```
* class/global variables
int arr[];
int size;
int cap;
```

```

if (x < size && arr[x] < arr[smallest])
    smallest = x;

```

```

if (smallest != i)
{
    swap(arr[i], arr[smallest]);
    MinHeapify(smallest);
}

```

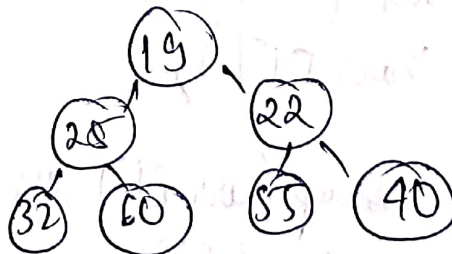
// This code find the smallest of the values of:
 // root, left & right child.

```

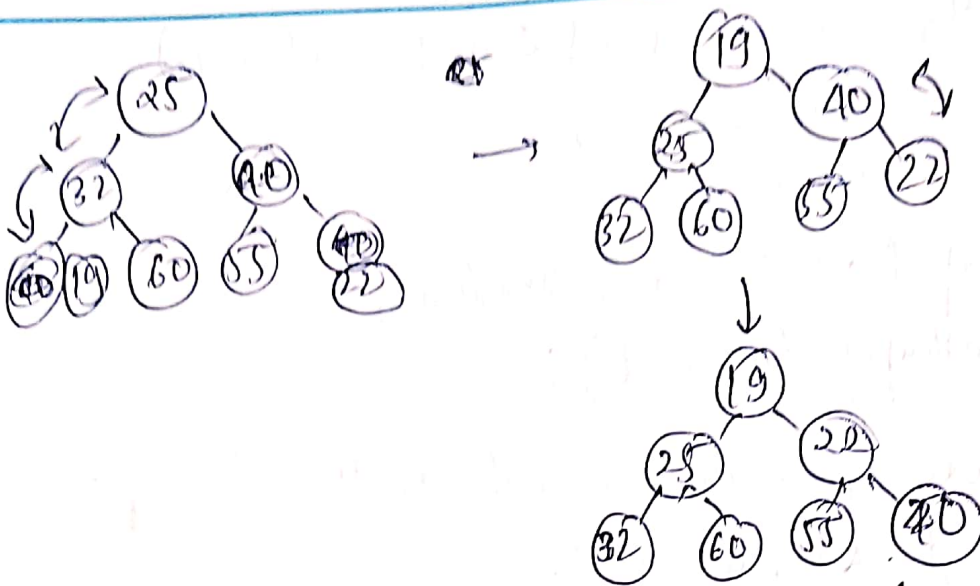
void heapsort(int arr[], int size)
{
    for (int i = size/2 - 1; i >= 0; i-- i--)
    {
        heapify(i);
    }
    for (int i = n-1; i >= 0; i--)
    {
        swap(arr[0], arr[i]);
        heapify(i);
    }
}

```

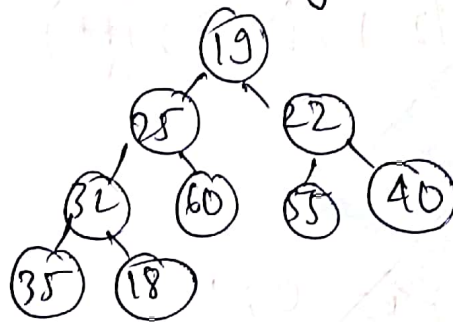
Implementing the following MinHeap as the following codes, we will get the output:-



working (P.T-O.)



(ii) ~~We will~~ To add 2 new ages we will append the 2 ages in the array & in the heap as :-



Now we insert in the Binary heap by :-

```
void insert(int x)
```

```
{
    if (size == capacity)
        return;
```

```
    size++;
```

```
    arr[size-1] = x;
```

```
    for (int i = size-1; i != 0 && arr[parent(i)] > arr[i];)
```

```
{
```

```
        swap(arr[i], arr[parent(i)]);
```

```
        i = parent(i);
```

```
}
```

```
}
```

Note : here $\rightarrow$ $\text{parent}(i) = (i-1)/2$
 $\text{left}(i) = (2*i+1)$
 $\text{right}(i) = (2*i+2)$

We will proceed to again heapify the table as follows:

